CSCI 565: Final Project Requirements: Fall 2025

DO NOT START IMPLEMENTING YOUR FINAL PROJECT UNTIL YOUR FINAL PROJECT PROPOSAL HAS BEEN SUBMITTED AND APPROVED.

Objectives

- Demonstrate your ability to create a new web application that you design from scratch, using HTML, CSS, Javascript, Bootstrap, Python, Django and Docker, deployed to Google Cloud Platform.

Application Requirements

- Define a web service that solves a problem in a domain that you are interested in. Be creative! Make something you would be proud to share. This project could make a nice addition to your school project portfolio, which will help you when you start your career search. Having a full-stack, hosted web service to share that you created from scratch will be well received by most prospective employers, even those not specifically seeking web developers.
- Avoid "generic" application domains like "an ecommerce website" or obvious knock-offs of existing services like Facebook…these are not unique or particularly interesting to employers. Pick something creative / unique that is related to a **specific** problem domain that you are interested in, perhaps something related to a hobby or other activity that you enjoy. Try to identify a service that solves a specific problem or fills a specific need, ideally something that doesn't exist or in common use already. Design an application that you would want to use.
- Scope the feature set of the final project appropriately for the time given (~5-6 weeks of development time, including for deployment) and the number of people on the team.
- The scope complexity may be more in the feature set of your solution, the front-end UI complexity / design, additional backend integrations, and/or in the way you deploy your application.
- An emphasis on feature set complexity would come from the features you define.
- An emphasis on UI complexity / design might come from using more advanced features of Bootstrap like grid for multi-column layout, targeting mobile devices, dynamic UI features using Javascript, or a more visually appealing UI design.
- An emphasis on back-end complexity might come from doing some kind of additional backend data processing (like for a recommender system), using an external **REST API**, publishing your own **REST API** using the **Django REST Framework, or** supporting user-uploaded content using Django's **Media** support for uploaded user content, which would also require deploying to GCP's **Bucket** service.
- You will submit a separate **Final Project Proposal** to get approval for your planned set of features , UI, components and deployment details.
- Use the Bootstrap front-end framework to generate a useable front-end UI.
- Use a Bootstrap navigation bar to provide the needed access to all of your application functionality.
- Include **Join**, **Login**, **Logout**, & **About** pages / functionality.
- Define at least **three** models related to your defined problem domain / solution (not counting the built-in User model). At least two of your defined models must reference other models you defined (using **ForeignKey**, **OneToOne**, or **ManyToMany** Django model fields).
- Deployment Requirements
    - You must use **Docker** to deploy your Final Project to **Google Cloud Platform** using the **Google Cloud Registry**.
    - You must deploy to a **Load Balancer** with a managed instance group with at least 2 app servers.
    - You must deploy to a **PostgresSQL** database service within **GCP**.
    - You must register a domain name and implement **SSL/HTTPS** support on your load balancer (I will show you how in lecture).
    - You must deploy **nginx** and **gunicorn** within your Docker image to maximize server capacity (I will show you how in lecture).
    - You must implement path **/server_info/** to output server and django configuration info, using the info provided here:
        - Install **Requests** Python Library:
            1. **pip install requests**
        - Add new dependency to **requirements.txt**

        - requests==2.22.0
    - Add this view function to one of your application views:
        1. Add to import section at top of one of your **views.py** files:
        2. **import requests**
        3. **from django.conf import settings**
        4. Add this view function:
        5. def server_info(request):
        6. server_geodata = requests.get('https://ipwhois.app/json/').json()
        7. settings_dump = settings.__dict__
        8. return HttpResponse("{}{}".format(server_geodata, settings_dump))
    - Create a path in **urls.py** to reference this new view, e.g. if you implemented the view section above in application **app1**:
        1. Add to import section at the top of **urls.py**:
        2. from app1 import views as app1_views
        3. Add to **urlpatterns** list:
        4. **path('server_info/', app1_views.server_info),**
- I will provide you with a github repository with codespaces enabled for you to use as your development environment.

Deliverables

See Assignments > Final Project > Final Project Deliverables.